Hash-Based Digital Signatures

Lamport OTS $\rightarrow$ WOTS $\rightarrow$ Merkle Signature Scheme

Aicha Slaitane Abdullah Algethami

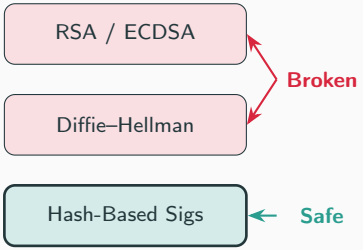
Applied Cryptography (CS 232) — KAUST

Outline

Theory & Implementation

The Quantum Threat

- **Shor's algorithm** (1994) breaks RSA, ECDSA, and Diffie–Hellman in polynomial time on a quantum computer.
- These schemes rely on the hardness of *factoring* or *discrete logs*.
- We need signatures based on different assumptions.



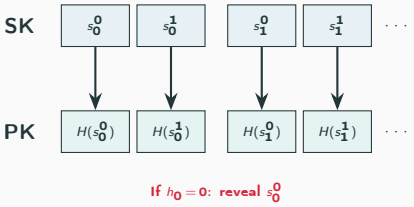
Hash-based signatures rely *only* on the one-wayness of hash functions (e.g. SHA-256).

- No number-theoretic assumptions.
- Grover's algorithm gives only a $\sqrt{}$ speedup $\Rightarrow$ SHA-256 still provides $\sim$128-bit quantum security.

Lamport One-Time Signature

Key Generation: *(Where H is a cryptographic collision resistant hash function e.g. SHA-256)*

- Generate 256 pairs of random 32-byte blocks.
- Secret key: $\mathrm{SK}[i] = (s_i^0,\ s_i^1)$
- Public key: $\mathrm{PK}[i] = \big(H(s_i^0),\ H(s_i^1)\big)$



Signing message m:

1. Compute $h = H(m)$ — 256 bits.
2. For each bit h_i, reveal $s_i^{h_i}$.

Verification:
Check $H(\sigma_i) \stackrel{?}{=} \mathrm{PK}[i][h_i]$ for all i.

Key Properties
Signature = 8,192 B | Signing: 0.03 ms (no hashing!) | Strictly one-time use

Why Is Lamport One-Time?

- Each signature reveals **256 of 512** secret blocks.
- A second signature on a different message reveals *different* blocks.
- After k reuses, an attacker has collected enough blocks to forge *any* message.

Reuses (k)	Blocks Exposed	Forgery Probability
1	50%	$\approx 0\%$
8	99.6%	38.5%
10	99.9%	78%
15	100%	$>99.9\%$

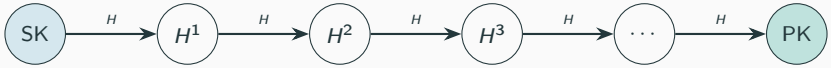
Two solutions: (1) Use WOTS to reduce sizes (2) Use Merkle trees to manage many keys

Winternitz OTS: Hash Chains

Key Idea: Group hash bits into base-w digits and use *hash chains* instead of single hashes.

What is a Hash Chain?

- A hash chain is formed by repeatedly applying a cryptographic hash function H to a secret starting value.
- Each step in the chain is one-way: you can compute forward easily, but computing backward requires inverting the hash.

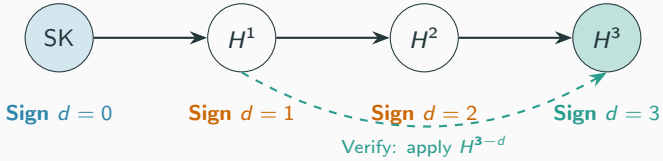


WOTS: Toy Example (Chain Length $w = 4$)

Suppose our message digit is $d \in \{0, 1, 2, 3\}$.

- **Secret Key (SK):** The random start of the chain.
- **Public Key (PK):** The end of the chain, $H^3(\mathrm{SK})$.

To sign the digit d, we reveal the intermediate value $\sigma = H^d(\mathrm{SK})$.



WOTS: Forgery Risk & Checksum

The Risk of Forgery:

- If an attacker sees $\sigma = H^d(\text{SK})$, they can easily compute $H(\sigma) = H^{d+1}(\text{SK})$.
- This means they can forge a signature for any digit $d' > d$.

The Solution: Checksum Logic

- We append a *checksum* to the message: $C = \sum(w - 1 - d_i)$.
- If an attacker tries to forge by increasing a message digit $d_i \to d_i'$, the checksum C must **decrease**.
- Decreasing a digit requires going *backward* in the checksum's hash chain, which means inverting the hash!

w	Sig Size (Bytes)	Chains
4	4,256	133
16	**2,144**	**67**
256	1,088	34

WOTS: Putting It All Together

The Full Winternitz OTS Scheme:

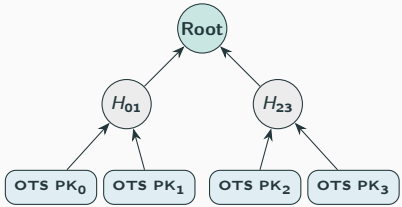
1. **Message Preparation:**
   - Hash the message M to get $h = H(M)$.
   - Split h into base-w digits: $d_1, d_2, \ldots, d_\ell$.
2. **Checksum Addition:**
   - Compute the checksum $C = \sum_{i=1}^{\ell}(w - 1 - d_i)$.
   - Convert C to base-w digits and append to the message digits.
   - Total digits $= \ell +$ checksum digits (each digit gets its own hash chain).
3. **Signing:**
   - For each digit v_i (message or checksum), compute $\sigma_i = H^{v_i}(\mathsf{SK}_i)$.
   - The full signature is the collection of all σ_i.
4. **Verification:**
   - The verifier computes the remaining steps in each chain: $H^{w-1-v_i}(\sigma_i)$.
   - The result must perfectly match the Public Key $\mathsf{PK}_i = H^{w-1}(\mathsf{SK}_i)$ for all chains.

From OTS/WOTS to the Merkle Signature Scheme

Both Lamport and WOTS are one-time — a fresh keypair is needed for every message.

Merkle's fix: pre-generate 2^h OTS keypairs and bind them under a single public key using a hash tree.

- Each OTS public key PK_i is hashed to form **leaf** L_i of the tree.
- Internal nodes are computed as $H(\text{left}\|\text{right})$.
- The **tree root** is the single MSS public key (32 B).
- To sign message i: run OTS_i and attach the auth path (h sibling nodes) to let anyone recompute the root.



Lamport or WOTS public keys as leaves

Advantage of WOTS over Lamport
WOTS ($w=16$) signatures are $\approx 4\times$ smaller (2 144 B vs. 8 192 B), directly shrinking the on-wire MSS signature.

Merkle Signature Scheme: Key Generation

Problem: One-Time Signatures (OTS) can only sign a single message securely.

Solution: Build a binary hash tree (Merkle Tree) over 2^h OTS public keys to sign 2^h messages.

Key Generation Process:

1. Generate 2^h independent OTS keypairs.
2. Hash each OTS public key to form the leaves: $\text{Leaf}_i = H(\text{OTS\_PK}_i)$.
3. Compute parent nodes by hashing the concatenation of children: $\text{Parent} = H(\text{left} \| \text{right})$.
4. The **MSS public key** is the tree root (just 32 B).

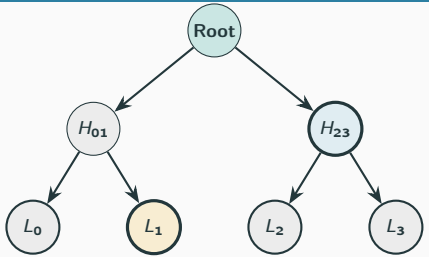
Merkle Signature Scheme: Sign & Verify

To sign the i-th message:

- Sign the message with OTS key i.
- Include the *auth path*: the h sibling hashes needed to reach the root.

To verify:

- Verify the underlying OTS signature against OTS_PK_i.
- Reconstruct the root from Leaf_i and the auth path.
- Check if reconstructed root $\stackrel{?}{=}$ MSS public key.



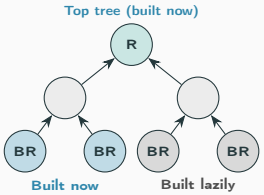
Signing leaf L_1:
Auth path $= [L_0,\ H_{23}]$
Verify: $H(H(L_0 \| L_1) \| H_{23}) = \text{Root}$

Multi-Layer Merkle Trees (Hypertrees)

Problem: Flat MSS KeyGen builds all 2^h OTS keys upfront — exponentially expensive for large h.

Idea: Stack two Merkle trees. The *top tree* is built immediately; each of its leaves certifies the root of a *bottom tree* that is built lazily only when needed.

- Total capacity: $N = 2^{h_{\text{top}}+h_{\text{bot}}}$ messages.
- Initial KeyGen: $O(2^{h_{\text{top}}} + 2^{h_{\text{bot}}})$ OTS keys.
- Signing produces *two* MSS signatures: one from the bottom tree, one from the top tree certifying the bottom root.
- This layering is the core idea behind **SPHINCS+ / SLH-DSA** (NIST post-quantum standard).



Evaluation & Extensions

Evaluation Setup: How Messages Are Generated

Two distinct message generation strategies are used in the evaluation:

Security Simulation (Key-Reuse Attack)

- A mix of **fixed commands** and **random strings**:
  - `"Attack at dawn"`, `"Hold the line"`...
  - `f"Random message {random.random()}"`
- 20 unique messages are signed with the *same* Lamport key.
- A fixed target (`"Retreat at noon"`) tests if a forgery is possible.
- Repeated over **200 independent trials** for statistical validity.

Performance Benchmarking

- A single fixed string is used across all timing trials:

```
"Post-quantum cryptography is exciting"
```

- Each configuration is measured over **5 repetitions**; the *median* is reported to filter noise.
- The same message is reused across all parameter sweeps for a fair, controlled comparison.

Lamport Key-Reuse Attack

Threat model: A *passive* attacker observes k signatures made with the **same** Lamport key.

- Each signature reveals one block per position (bit 0 or 1).
- After k signatures, the probability a specific block is *still hidden* is $(1/2)^k$.

Forgery probability:

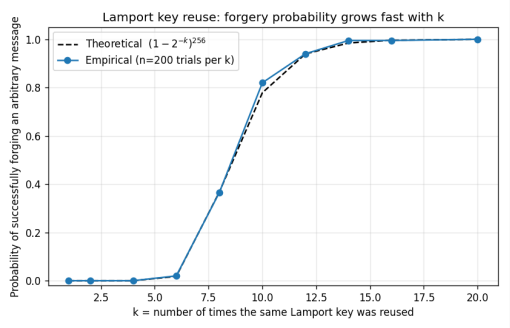
$$P_{\text{forge}}(k) = \left(1 - 2^{-k}\right)^{256}$$

The attacker needs all 256 blocks matching the target message's hash bits. This converges to certainty exponentially.

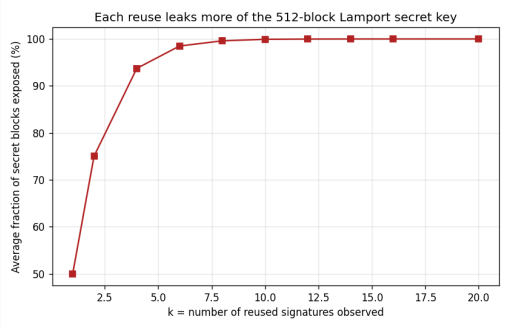
k	P_{forge}	**SK Exposed**
1	$\approx 0\%$	50%
5	0.03%	96.9%
8	38.5%	99.6%
10	78%	99.9%
12	94%	100%
15	99.8%	100%
20	100%	100%

Results from 200 independent trials per k.

Empirical Validation



Empirical vs. theoretical forgery rate.
Theory and experiment match closely.



Fraction of SK exposed vs. reuse count.
By $k = 12$ the complete key is recovered.

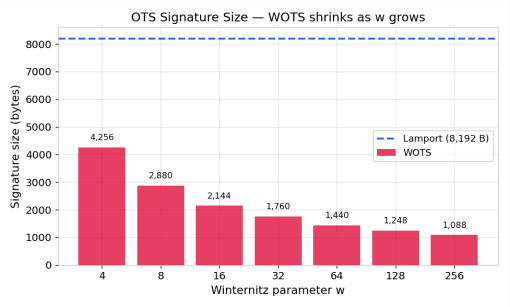
OTS Performance Comparison

Scheme	KG (ms)	Sign (ms)	Verify (ms)	Sig (B)	PK (B)
Lamport	0.17	0.03	0.17	8,192	16,384
WOTS $w=4$	0.29	0.15	0.15	4,256	4,256
WOTS $w=16$	0.56	0.30	0.28	2,144	2,144
WOTS $w=64$	1.49	0.68	0.84	1,440	1,440
WOTS $w=256$	4.47	2.28	2.29	1,088	1,088

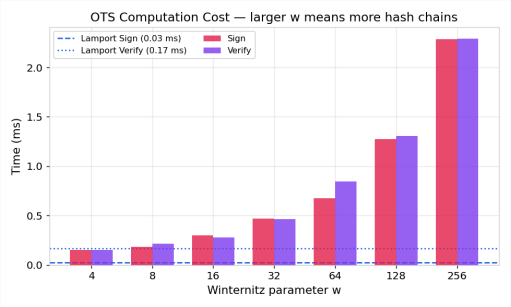
Key observations:

- Lamport signing is extremely fast (0.03 ms) — no hash computation, only copying blocks.
- WOTS $w=256$ is **7.5× smaller** but **75× slower** than Lamport.
- $w=16$ is the practical sweet spot — adopted by XMSS and SPHINCS+.

OTS: Size vs. Speed Trade-off



Signature size decreases *logarithmically* with w.



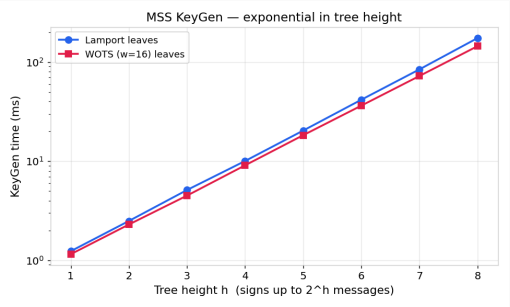
Computation time grows *linearly* — each chain is $w-1$ hashes long.

MSS Performance: Scaling with Tree Height

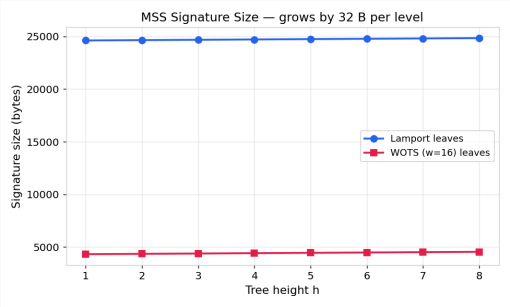
Height h	Messages	KG (ms)	Sign (ms)	Sig Size (B)
Lamport leaves				
4	16	5.70	0.24	24,640
6	64	22.63	0.26	24,704
8	256	92.52	0.27	24,768
WOTS ($w=16$) leaves				
4	16	9.08	0.53	4,420
6	64	36.31	0.55	4,484
8	256	144.46	0.58	4,548

- KeyGen: $O(2^h)$ — doubles with each additional level.
- Sign/Verify: **Constant** regardless of tree height.
- Signature overhead: only **+32 B per level** (one SHA-256 sibling).

MSS: KeyGen is the Bottleneck

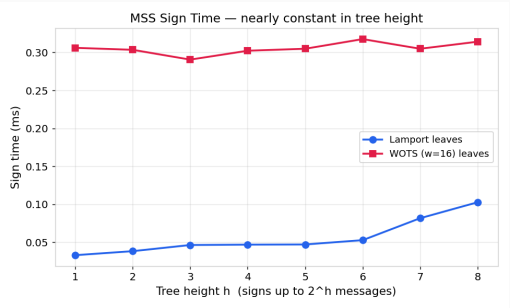


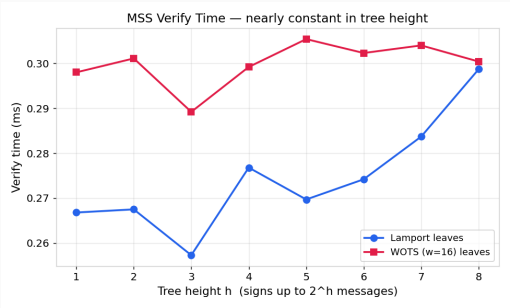
KeyGen scales as $O(2^h)$: the log-scale plot shows a linear trend.



Signature size grows by just 32 B per tree level.

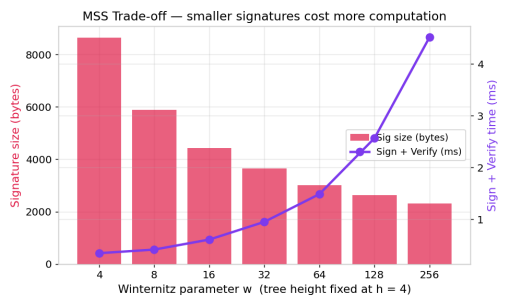
MSS: Sign & Verify are Constant-Time in h





The h additional hashes for the auth path are negligible compared to the underlying OTS operations.

Parameter Tuning: MSS with Varying w



Fixed: $h = 4$ (16 messages).

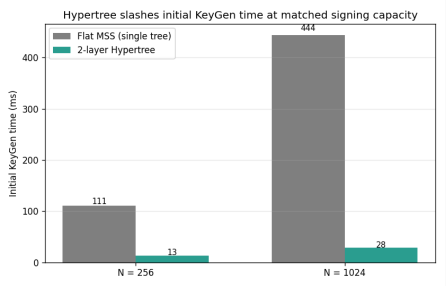
The *knee* near $w = 16$ marks the optimal balance:

- Below $w = 16$: rapid size improvement with modest time cost.
- Above $w = 16$: diminishing size returns, escalating time cost.

Recommendation
$w = 16$, $h = 4$–8
This matches the XMSS and SPHINCS+ standard configurations.

Multi-Layer Merkle Trees (Hypertrees): Performance

Scheme	N	KG (ms)	Sig (B)
Flat MSS, $h=8$	256	111.1	4,548
Hypertree, $h_t=h_b=4$	256	**13.5**	8,876
Flat MSS, $h=10$	1024	443.7	4,612
Hypertree, $h_t=h_b=5$	1024	**28.5**	8,940



Result: 8.2× faster KeyGen at $N=256$, 15.6× faster at $N=1024$. Signature roughly doubles.

Conclusion

1. **Implemented** Lamport OTS, WOTS, Merkle Signature Scheme, and a 2-layer Hypertree from scratch using SHA-256.
2. **Benchmarked** across the full (w, h) parameter space — confirmed theoretical scaling predictions.
3. **Demonstrated** the key-reuse vulnerability with 200-trial Monte Carlo experiments matching the theoretical model.
4. **Extended** flat MSS to a 2-layer hypertree — the same trick used by SPHINCS+/SLH-DSA — for an order-of-magnitude KeyGen speedup at matched capacity.

Key Insights

- WOTS $w = 16$ is the standard for a reason: $4\times$ smaller than Lamport at manageable cost.
- Flat MSS KeyGen is $O(2^h)$, but sign/verify are constant and signatures grow by only 32 B/level.
- Hypertrees turn the $O(2^h)$ KeyGen cost into $O(\sqrt{N})$ at the cost of $\sim 2\times$ signature size.
- Hash-based signatures are a viable, well-understood path to post-quantum security.

Thank You

Questions?

Aicha Slaitane — aicha.slaitane@kaust.edu.sa
Abdullah Algethami — abdullah.algethami@kaust.edu.sa